

Experiment Flow - Working Guide

Unofficial companion: how we actually run the study - data, commands, results. Keep this next to the keyboard.

1. One-time setup

```
cd research_ievf
pip install -r requirements.txt
pip install fpdf2          # for the PDF reports
copy .env.example .env    # then fill in the keys
```

- Model keys: OPENAI_API_KEY, ANTHROPIC_API_KEY, GOOGLE_API_KEY, DEEPSEEK_API_KEY, DASHSCOPE_API_KEY (Qwen), MOONSHOT_API_KEY (Kimi).
- Judge: JUDGE_API_KEY (token only - the judge model and endpoint are set in config.yaml under judges:, default deepseek-chat). The judge must be a DIFFERENT family from the model being graded.
- MoReBench (morebench/morebench on HF) is public: HF_TOKEN in .env is optional, only needed to raise your rate limit.

2. Data - where everything comes from

- MoReBench: run `python fetch_morebench.py` once -> saves `data/morebench_public.csv` and `morebench_theory.csv`. Works offline afterwards.
- SimpleBench: `data/simple_bench_public.json` - already in the repo, nothing to do.
- AschBench (ours): sources live in `benchmark/items/*.jsonl`; the single release file is `data/asch_bench_public.json`. After editing any item, rebuild + re-validate with `python benchmark/build_release.py`.
- The fetch-data step normalizes all benchmarks into one file: `data/processed/items.jsonl`.

3. The flow - first to last

Step 0 costs nothing and tests all plumbing with fake answers. Always run it first.

```
python run_pipeline.py --phase fetch-data --simplebench --aschbench
python run_pipeline.py --phase baseline --dry-run
python run_pipeline.py --phase exp1 --dry-run
python run_pipeline.py --phase exp2 --dry-run
python run_pipeline.py --phase exp1 --dry-run --mit
python run_pipeline.py --phase exp2 --dry-run --mit
```

Then the real run (this spends API money):

```
python run_pipeline.py --phase fetch-data --simplebench --aschbench
python run_pipeline.py --phase baseline
python run_pipeline.py --phase exp1
python run_pipeline.py --phase exp2
python run_pipeline.py --phase analyze
python run_pipeline.py --phase exp1 --mit
python run_pipeline.py --phase exp2 --mit
python run_pipeline.py --phase analyze
python make_reports.py
```

- `--models qwen,deepseek` runs only some models; `--limit 5` runs a few items. Use both for cheap tests.
- Everything is resume-safe: if a run stops, re-run the same command; finished items are skipped automatically.

4. Where to see results

- logs/results.db - every response and every YES/NO grade. Quick health check: `python check_setup.py`
- reports/metrics_by_condition.csv - the 4 bias numbers per condition (P1..P6, G_k2..G_k4).
- reports/metrics_by_model.csv - which model caves most.
- reports/metrics_by_benchmark.csv - AschBench vs MoReBench side by side (must tell the same story).
- reports/hysteresis.csv - stuck rate per group size after pressure is removed.
- reports/gate_decisions.csv - what the IEVF gate allowed or blocked, and why.
- reports/before_after.csv - IEVF off vs on: the paper's main table.
- reports/figures/*.png - dose_response.png, before_after.png, per_model.png.
- reports/report_1_pipeline.pdf + report_2_experiment_flow.pdf - the two readable reports (this script).

5. Rules that keep the science clean

- Dry-run first, always. It is free.
- Freeze items and prompts once the real run starts; temperature stays 0.
- Never let the judge be the same model family as the target.
- If anything crashes: just re-run the same command.